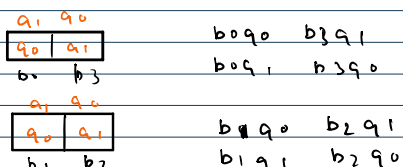
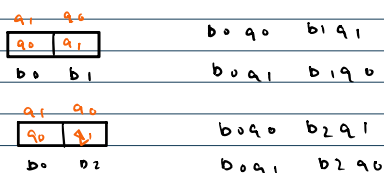
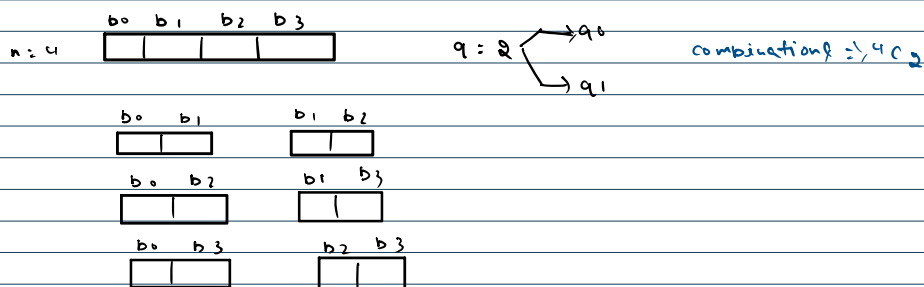


It is a technique that incrementally builds solutions to a problem and abandoning (backtracking) a solution as soon as it determines that the solution cannot be completed to a valid one.

Queen Permutations on a 1D Board

Input :- $n \Rightarrow$ no. of boxes
 $q \Rightarrow$ no. of queens

output :- all valid per. of placing q queens in n boxes



b₀ b₃

a ₁	a ₀
q ₀	q ₁
b ₁	b ₂

q ₁	q ₀
q ₀	q ₁
b ₁	b ₃

q ₀	q ₀
q ₀	q ₁
b ₂	b ₃

b₁ q₀ b₂ q₁

b₁ q₁ b₂ q₀

b₁ q₀ b₃ q₁

b₁ q₁ b₃ q₀

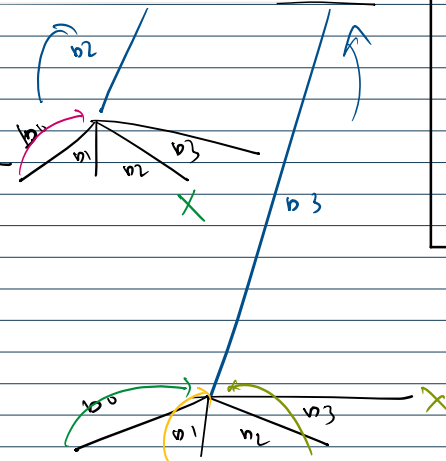
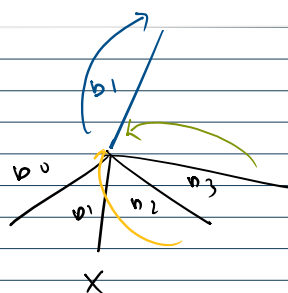
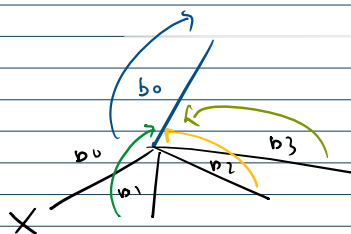
b₂ q₀ b₃ q₁

b₂ q₁ b₃ q₀

Permutations $\therefore$ Comb $\times n!$ $\therefore$ $\frac{n!}{(n-r)!}$ $\times r!$

$\downarrow$
 $nCr \times r!$
 $\frac{n!}{(n-r)!} \times r!$
 $= n!$

$nPr = \frac{n!}{(n-r)!}$

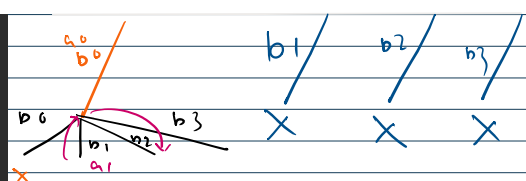


q ₁	q ₀
q ₀	q ₁
b ₀	b ₁
q ₁	q ₀
q ₀	q ₁
b ₀	b ₂
q ₁	q ₀
q ₀	q ₁
b ₀	b ₃

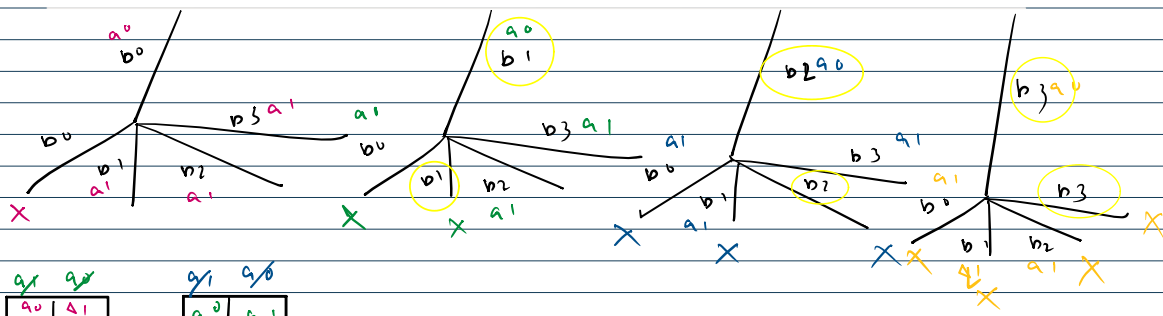
q ₀	q ₁	q ₁	q ₁
b ₀	b ₁	b ₂	b ₃

$\left\{ \begin{array}{l} b_0 q_0 \ b_1 q_1 \\ b_0 q_1 \ b_2 q_1 \\ b_0 q_0 \ b_3 q_1 \end{array} \right.$

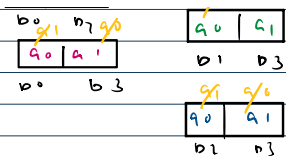
```
static void perm(boolean[] board, int tq, int qpsf, String ans) {
    if (qpsf == tq) {
        System.out.println(ans);
        return;
    }
    for (int i = 0; i < board.length; i++) {
        if (board[i] == false) {
            board[i] = true; // queen placed
            perm(board, tq, qpsf + 1, ans + " b" + i + " q" + qpsf);
        }
    }
}
```



Queen Combinations on a 1D Board



q ₁	q ₀
q ₀	q ₁
b ₀	b ₁
q ₀	q ₁
b ₀	b ₂
q ₁	q ₀
q ₀	q ₁
b ₀	b ₃



coin change - permutations

coins $\rightarrow$ 1 2 4

Argument $\rightarrow$ 6

o/p :-

1 1 1 1 1

1 1 1 1 2

1 1 2 2

2 2 2

1

1

1

l

9.9

25

22

11 2 12

